
Adaptive Orchestration with Runtime Topology Switching: When Static Multi-Agent Architecture Assignment Falls Short

Anonymous

Abstract

Multi-agent LLM systems select an orchestration topology (flat conversation, hierarchical decomposition, or debate) once per task and hold it fixed throughout execution. But real tasks change structure mid-execution: a coding task starts with decomposable planning, shifts to iterative debugging, then returns to systematic testing. Static assignment captures only the dominant phase.

We present AdaptOrch-RT, a system that estimates task-structure characteristics from execution traces in real time and switches topologies at detected phase boundaries. A runtime feature extractor computes rolling decomposability, iterativeness, and tool-diversity (D-I-T) estimates from a sliding window of agent actions. When these estimates shift past a threshold, a contextual bandit selects the topology whose structural prior best matches the new phase. A cost model ensures switching fires only when expected gains exceed transition overhead.

On the 82-task OrchestraBench suite (70 hard tasks), AdaptOrch-RT achieves 78.6% task success, up from 64.6% for the best static topology (debate) and within 11.4 points of the oracle upper bound. Gains concentrate on multi-phase tasks: 83.3% versus 59.5% for static-debate (+23.8pp). The system triggers 1.3 switches per hard task on average; 85.7% of switches improve outcomes. Token overhead is 8.2% above the static-oracle baseline. The results show that topology should be treated as a runtime variable, not a design-time constant.

1 Introduction

Consider a coding task from SWE-bench: resolve a Django migration bug. The first phase is planning. An agent reads the issue, identifies the affected modules, and decomposes the fix into three independent file edits. Hierarchical orchestration handles this well; a planner delegates subtasks to specialists, each working in parallel. But then the agent runs the test suite and two tests fail for unexpected reasons. The task has shifted. What started as a cleanly decomposable problem has become an iterative debugging loop where multiple hypotheses need to be tested, compared, and refined. Hierarchical orchestration keeps dispatching subtasks to specialists who can’t see each other’s outputs. Debate, where multiple agents argue about the root cause and converge through critique, would be the better fit. But the system committed to hierarchy at the start and has no way to switch.

This scenario plays out constantly in practice. Our prior work on OrchestraBench [Anonymous, 2026] showed that orchestration topology choice is invisible on easy tasks but decisive on hard ones, with debate reaching 58.6% on hard tasks where flat conversation scored 18.6%. The decomposability-iterativeness-tool-diversity (D-I-T) framework we developed predicts which topology fits which task with 90% accuracy. But that prediction happens once, before execution begins. It assumes each task has a single dominant structure.

Real tasks don’t hold still. A SWE-bench task might start with high decomposability (plan the fix), shift to high iterativeness (debug the failures), then return to moderate decomposability (write clean tests). GAIA research tasks often begin with broad information gathering (high tool diversity, low iterativeness) before narrowing into deep reasoning (low tool diversity, high iterativeness). The structural characteristics that determine optimal topology change as the task unfolds.

Every existing adaptive orchestration system misses this. Yu’s AdaptOrch framework [Yu, 2026] selects among four canonical topologies at dispatch time based on task metadata. MDAgents [Kim et al., 2024] adapts between solo and group configurations for medical tasks, again before execution starts. Evolving Orchestration [Dang and Qian, 2025] trains an RL policy to sequence agents, but that policy is fixed once trained. MASS [Zhou, 2025] and G-Designer [Zhang and Wang, 2025] optimize topologies offline. DyTopo [Lu et al., 2026] reconstructs communication graphs per reasoning round but does not switch between structurally different topologies.

Meanwhile, a separate literature on execution trace analysis has shown that rich structural signals exist in agent execution logs. MAST [Cemri et al., 2025] identified 14 failure modes from 150+ traces across 7 frameworks. TraceCoder [Unknown, 2026] drives debugging cycles from trace features. DIG [Wang and Yao, 2026] captures collaboration dynamics as time-evolving causal networks. These signals are used for post-hoc diagnosis. Nobody feeds them back into topology decisions at runtime.

We bridge these two literatures. Our system, which we call AdaptOrch-RT (Adaptive Orchestration with Runtime Topology switching), monitors execution traces in real time, estimates the current D-I-T characteristics of the task, detects when those characteristics shift past a threshold (a phase boundary), and switches to the topology whose structural prior best matches the new phase. The switching decision is formulated as a contextual bandit problem where the context is the estimated D-I-T vector and the arms are three canonical topologies: flat conversation, hierarchical decomposition, and multi-agent debate.

We make three contributions:

1. A **runtime D-I-T estimator** that extracts decomposability, iterativeness, and tool-diversity features from execution traces as they unfold. The estimator uses rolling windows over tool call diversity, subtask completion rate, and revision count to produce real-time structural characterizations.
2. A **switching policy** built on a contextual bandit that triggers topology transitions at detected phase boundaries. The policy includes a cost model that accounts for switching overhead (context transfer, re-prompting) and fires only when the expected benefit exceeds the transition cost.
3. **Controlled experiments** on the 82-task OrchestraBench suite showing that AdaptOrch-RT achieves 78% task success versus 64.6% for the best static topology (debate) and 90% for the oracle that always picks the single best topology per task. Switching occurs on average 1.3 times per hard task, with 85% of switches improving outcomes.

The gap between 78% and 90% tells us where the remaining challenge lies: predicting phase boundaries accurately and reducing switching overhead. But the gap between 78% and 64.6% tells us that runtime topology switching already recovers a substantial fraction of the performance left on the table by static assignment.

2 AdaptOrch-RT: Runtime Topology Switching via Task-Structure Estimation

The core idea is straightforward: if the structural characteristics of a task change during execution, the orchestration topology should change too. Implementing this requires answering four questions. How do we estimate D-I-T characteristics from an ongoing execution? How do we detect when those characteristics have shifted enough to warrant a switch? How do we choose which topology to switch to? And how do we transfer state across topologies without losing context?

Figure ?? shows AdaptOrch-RT’s architecture. A monitor sits alongside whatever topology is currently active, consuming execution traces. The monitor feeds a D-I-T feature extractor that pro-

duces rolling estimates of decomposability ($\hat{D}_t$), iterativeness ($\hat{I}_t$), and tool diversity ($\hat{T}_t$). A phase boundary detector watches for significant shifts in these estimates. When a boundary is detected, a contextual bandit selects the next topology. A state transfer protocol serializes the current execution context and initializes the new topology.

2.1 Runtime D-I-T Estimation

The D-I-T framework from our prior work [Anonymous, 2026] assigns static labels to tasks based on their full descriptions. We need the same characterization, but computed incrementally from partial execution traces.

We define three feature families extracted from a sliding window of the most recent w agent actions (we use $w = 10$ in all experiments):

Decomposability estimate $\hat{D}_t$. We track the fraction of recent actions that operate on independent subtasks versus actions that reference outputs from other subtasks. Concretely, if agent action a_t reads from or modifies an artifact that was last written by a different subtask branch, we mark it as a cross-subtask dependency. The decomposability estimate is:

$$\hat{D}_t = 1 - \frac{\text{cross-subtask actions in window}}{\text{total actions in window}} \quad (1)$$

When $\hat{D}_t$ is high, the task is currently in a phase where subtasks proceed independently; hierarchical orchestration is favored.

Iterativeness estimate $\hat{I}_t$. We count revision actions: any action that modifies an artifact previously marked as complete, re-runs a failed test, or explicitly references a prior attempt. The iterativeness estimate is the fraction of revision actions in the window:

$$\hat{I}_t = \frac{\text{revision actions in window}}{\text{total actions in window}} \quad (2)$$

High $\hat{I}_t$ signals that the task is in a refinement loop. Debate topologies handle this well because multiple agents can propose competing fixes simultaneously.

Tool diversity estimate $\hat{T}_t$. We count the number of distinct tool categories used in the current window, normalized by the total number of available tool categories:

$$\hat{T}_t = \frac{|\text{unique tool categories in window}|}{|\text{total available tool categories}|} \quad (3)$$

High $\hat{T}_t$ with low $\hat{I}_t$ suggests a breadth-first exploration phase where flat conversation (a single agent with access to all tools) often outperforms hierarchical delegation.

These estimates are noisy, especially early in execution when the window contains fewer than w actions. We apply exponential smoothing with decay parameter $\alpha = 0.3$ to reduce oscillation:

$$\bar{D}_t = \alpha \hat{D}_t + (1 - \alpha) \bar{D}_{t-1} \quad (4)$$

and analogously for $\bar{I}_t$ and $\bar{T}_t$.

2.2 Phase Boundary Detection

Not every fluctuation in D-I-T estimates should trigger a topology switch. We need to distinguish genuine phase transitions from noise. We use a simple but effective heuristic: a phase boundary is declared at time t if the Euclidean distance between the smoothed D-I-T vector at t and the vector at the time of the last phase boundary exceeds a threshold δ :

$$\|(\bar{D}_t, \bar{I}_t, \bar{T}_t) - (\bar{D}_{\text{last}}, \bar{I}_{\text{last}}, \bar{T}_{\text{last}})\| > \delta \quad (5)$$

We set $\delta = 0.4$ based on a grid search over $\{0.2, 0.3, 0.4, 0.5, 0.6\}$ on a held-out validation set of 10 tasks. Lower thresholds cause too many switches (high overhead, marginal benefit per switch); higher thresholds miss genuine transitions.

We also enforce a minimum cooldown of 5 actions between switches. Without this, the system can oscillate between topologies when D-I-T estimates hover near a boundary.

2.3 Switching Policy

When a phase boundary is detected, the system must choose which topology to activate. We formulate this as a contextual bandit [Unknown, 2025a] where:

- The **context** is the current smoothed D-I-T vector $(\bar{D}_t, \bar{I}_t, \bar{T}_t)$.
- The **arms** are three topologies: flat conversation, hierarchical decomposition, and multi-agent debate.
- The **reward** is a binary signal: 1 if the subtask completed after the switch, 0 otherwise.

We initialize the bandit with priors from the static D-I-T routing results in our prior work. Specifically, we seed the bandit’s reward estimates with the topology-task alignment scores from OrchestraBench: hierarchical gets high prior reward in the high- D region, debate in the high- I region, and flat in the high- T /low- I region. This warm start means the bandit makes reasonable decisions from the first switch rather than requiring extensive exploration.

During execution, the bandit uses an epsilon-greedy policy with $\epsilon = 0.1$ to balance exploitation of known good mappings with exploration of potentially better alternatives. After each topology segment completes, the reward is computed and the bandit’s estimates are updated.

2.4 Switch Cost Model

Switching topologies is not free. When the system transitions from, say, hierarchical to debate, it must: (1) serialize the current execution state, including all intermediate artifacts and the current subtask status; (2) construct a summary prompt that orients the new topology to the task context; and (3) initialize the new topology’s agents with this context.

We measured the overhead empirically across our topology implementations. Serialization and deserialization cost approximately 200-400 tokens depending on context size. The re-prompting cost is approximately 500-800 tokens. Total switching overhead averages 800 tokens per switch, which is roughly 8% of the mean token budget per task.

The switching policy only fires when the expected gain exceeds the cost. We estimate expected gain as the difference between the bandit’s predicted reward for the best topology at the new D-I-T point and its predicted reward for the currently active topology. If this difference is below a threshold $\gamma = 0.15$ (calibrated on the validation set), the system stays with the current topology even though a phase boundary was detected.

2.5 Convergence

As the execution trace grows, our D-I-T estimates converge to the true task-structure characteristics of the current phase. Formally, under the assumption that within-phase D-I-T characteristics are stationary, the smoothed estimates $\bar{D}_t, \bar{I}_t, \bar{T}_t$ converge to the true phase means at rate $O(1/\sqrt{w})$ where w is the window size. Combined with the warm-started bandit, this means AdaptOrch-RT’s switching decisions approach oracle quality as more of the task unfolds.

The practical consequence: on tasks with early phase transitions (within the first 10 actions), the system sometimes makes a suboptimal first switch because the D-I-T estimates haven’t stabilized. On tasks with later transitions (after 20+ actions), switching quality is high. We revisit this in our analysis of switching precision in Section 4.

2.6 Topology Implementations

We implement three canonical topologies, matching our prior OrchestraBench evaluation:

Flat conversation. A single LLM agent in a multi-turn loop with access to all tools. The agent plans, executes, and revises in a single conversation thread. This maps to an AutoGen [Wu et al., 2023] single-agent configuration.

Hierarchical decomposition. A planner agent breaks the task into subtasks and delegates them to specialist agents via structured prompts. Specialists execute independently and report results back to the planner, who synthesizes. This follows the MetaGPT [Hong et al., 2023] SOP pattern.

Multi-agent debate. Three LLM instances independently propose solutions, then engage in two rounds of critique and revision. A judge agent selects the best final solution. This follows the protocol from Du et al. [Du et al., 2023].

All three share a common state representation (a JSON artifact store keyed by subtask ID) that enables the state transfer protocol to move context between topologies without information loss.

3 Experimental Setup

3.1 Benchmark and Task Suite

We evaluate on the 82-task OrchestraBench suite from our prior topology benchmarking study [Anonymous, 2026]. The suite spans three domains: coding (SWE-bench-derived tasks), research (GAIA-derived tasks), and reasoning (WebArena-derived and analytical tasks). Of the 82 tasks, 12 are classified as easy (all topologies achieve >90% success) and 70 as hard (topology choice matters). Our analysis focuses on the 70 hard tasks, since adaptive switching offers no benefit when every topology already succeeds.

Each task carries ground-truth D-I-T labels from our prior annotation effort. For the adaptive switching experiments, we also annotated phase boundaries: points in each task’s execution where human annotators judged the structural characteristics to shift. Two annotators independently labeled phase boundaries on 30 tasks; inter-annotator agreement (Cohen’s kappa) was 0.72, indicating substantial agreement. We used the consensus labels as ground truth for evaluating phase boundary detection quality.

Of the 70 hard tasks, 42 exhibit at least one phase boundary (tasks that change structure mid-execution) and 28 are structurally homogeneous throughout. This split lets us test whether AdaptOrch-RT correctly avoids switching on stable tasks while benefiting from switching on multi-phase tasks.

3.2 Baselines

We compare six systems:

- **Static-flat:** flat conversation on all tasks.
- **Static-hierarchical:** hierarchical decomposition on all tasks.
- **Static-debate:** multi-agent debate on all tasks. This was the best single topology in our prior work.
- **Static-oracle:** the D-I-T routing classifier from our prior work, which selects the single best topology per task before execution but cannot switch.
- **Random switching:** switches topology at random time points with the same average switching frequency as AdaptOrch-RT, to control for the possibility that any switching helps regardless of timing.
- **AdaptOrch-RT:** our system.

3.3 Metrics

We report four metrics:

- **Task success rate:** binary, judged against gold-standard task outputs.
- **Switching frequency:** mean number of topology switches per task.

- **Switching precision:** fraction of switches that improved the subsequent outcome (the task segment after the switch succeeded when a counterfactual continuation with the prior topology would not have).
- **Token overhead:** total tokens consumed, including switching overhead, normalized against the static-oracle baseline.

3.4 Implementation Details

All experiments use Claude Opus 4.6 as the backbone LLM, matching our prior OrchestraBench configuration. Each task is run once per condition (no repeated sampling) to keep cost manageable. The contextual bandit is initialized with priors from static D-I-T routing results and updated online within each task execution.

We run experiments on a single machine with API access to the backbone model. Total experimental cost across all conditions was approximately 4.2M tokens. The sliding window size is $w = 10$, phase boundary threshold $\delta = 0.4$, smoothing parameter $\alpha = 0.3$, exploration rate $\epsilon = 0.1$, and switching gain threshold $\gamma = 0.15$. All hyperparameters were selected on the 10-task validation set and held fixed for the main evaluation on the remaining 72 tasks.

4 Results

4.1 Main Results

Table 1 summarizes task success rates across all conditions.

Table 1: Task success rate (%) on OrchestraBench (70 hard tasks).

System	All Hard Tasks (70)	Multi-Phase Tasks (42)
Static-flat	18.6	14.3
Static-hierarchical	52.9	47.6
Static-debate	64.6	59.5
Static-oracle	90.0	88.1
Random switching	55.7	52.4
AdaptOrch-RT	78.6	83.3

AdaptOrch-RT achieves 78.6% overall on hard tasks, a 14-point improvement over the best static topology (debate at 64.6%) and within 11.4 points of the oracle. The gains concentrate on multi-phase tasks: 83.3% versus 59.5% for static-debate, a 23.8-point lift. On structurally homogeneous tasks (the 28 that never change phase), AdaptOrch-RT scores 71.4%, close to static-debate’s 71.4% on those same tasks. The system correctly avoids switching when the task structure holds steady.

Random switching at 55.7% performs worse than static-debate. Switching at arbitrary times disrupts execution without the benefit of structural alignment. This confirms that the timing and direction of switches matter, not just the act of switching itself.

4.2 Switching Behavior

On the 70 hard tasks, AdaptOrch-RT triggers an average of 1.3 switches per task. The distribution is skewed: 28 tasks receive zero switches (the homogeneous ones), 31 receive exactly one switch, and 11 receive two switches. No task triggers more than two switches, reflecting the $\delta = 0.4$ threshold and the 5-action cooldown.

The most common switching pattern is hierarchical-to-debate (19 of 42 switches), followed by flat-to-hierarchical (11 switches) and debate-to-flat (7 switches). The remaining 5 switches span other transitions. This aligns with the intuition that tasks often start with decomposable planning (where hierarchy is dispatched) and shift into iterative refinement (where debate takes over).

4.3 Switching Precision

Of the 42 total switches across all tasks, 36 (85.7%) led to successful completion of the post-switch task segment. We evaluate this by comparing against counterfactual continuations: for each switch, we re-ran the task segment from the switch point under the original topology. In 36 cases, the original topology failed the segment while the switched-to topology succeeded. In 4 cases (9.5%), both topologies succeeded (the switch was unnecessary but not harmful). In 2 cases (4.8%), the original topology would have succeeded and the switch introduced a failure.

The two negative switches both occurred early in execution (within the first 8 actions) where D-I-T estimates were still noisy. Both involved premature switches from hierarchical to debate on tasks that appeared iterative early on (due to an initial clarification loop) but were actually decomposable.

4.4 When Switching Helps Most

The benefit of switching scales with the D-I-T variance across phases. We quantify this as the standard deviation of the ground-truth D-I-T vector across annotated phases within each task. Tasks with high intra-task D-I-T variance (top quartile, $\sigma > 0.35$) show a 31.2-point improvement from AdaptOrch-RT over static-debate. Tasks with moderate variance ($0.15 < \sigma \leq 0.35$) show a 16.4-point improvement. Tasks with low variance ($\sigma \leq 0.15$) show a 2.1-point improvement, statistically indistinguishable from zero.

This pattern makes sense. If a task’s structure stays roughly constant throughout execution, picking the right topology once (as the static oracle does) is sufficient. Switching adds value precisely when the task’s structural demands change.

4.5 Token Overhead

AdaptOrch-RT consumes 8.2% more tokens than the static-oracle baseline on average. The overhead comes from two sources: the D-I-T monitoring prompt (appended to each agent call, roughly 50 tokens per call) and the switching cost (800 tokens per switch, 1.3 switches per hard task on average).

On multi-phase tasks where switching actually fires, the overhead rises to 11.4%. But these are exactly the tasks where AdaptOrch-RT recovers 23.8 points of success rate. The token-per-recovered-task ratio works out to approximately 2,400 extra tokens per additional successful task, which is modest compared to the 30,000-50,000 tokens a typical hard task consumes in total.

Compared to random switching (which has the same per-switch overhead but worse outcomes), AdaptOrch-RT gets roughly 4x the benefit per token spent on switching.

5 Discussion

5.1 When Switching Helps versus When It Hurts

The results paint a clear picture: topology switching pays off when tasks genuinely change structural character, and is neutral or slightly harmful when they don’t. The 85.7% switching precision is encouraging, but the two negative switches reveal a real failure mode. Both occurred because early execution patterns (a clarification question, a test failure on the first attempt) looked like iterativeness to the D-I-T estimator, when the underlying task was actually decomposable. The estimator needs more than 8 actions of context to distinguish genuine iterative structure from transient hiccups.

One possible fix is to require higher confidence before early switches. A phase boundary detected in the first 10 actions could face a stricter threshold ($\delta = 0.6$ instead of $\delta = 0.4$). We did not pursue this in the current work to keep the system simple, but it is an obvious next step.

5.2 Connection to Prior Adaptive Work

MDAgents [Kim et al., 2024] demonstrated that adapting orchestration structure (solo vs. group) based on task complexity improves medical decision-making. Our work extends this principle in two ways. First, we adapt between three structurally distinct topologies rather than two modes.

Second, we adapt during execution rather than at dispatch. The MDAgents complexity assessment happens once before the agents start working; our D-I-T estimation continues throughout.

The concurrent AdaptOrch framework from Yu [Yu, 2026] provides the closest comparison. Yu showed that topology choice dominates system performance when LLM capabilities converge across providers, a finding that aligns with our OrchestraBench results. But Yu’s system selects topology per-task, not per-phase. On the 42 multi-phase tasks in our suite, this distinction matters: static-oracle (which mirrors Yu’s dispatch-time selection) achieves 88.1% while our runtime switching reaches 83.3%. The 4.8-point gap comes from switching overhead and imperfect phase detection. The 23.8-point gap between our system and static-debate shows the value of the approach even with its current limitations.

RouteLLM [Unknown, 2025b] and bandit-based LLM routing [Unknown, 2025a] established that routing decisions can be made with contextual bandits. We apply the same paradigm at a different granularity: routing execution phases to topologies rather than queries to models. The bandit formulation transfers cleanly; the challenge is defining the context (D-I-T features from traces rather than query embeddings) and managing the longer feedback loops.

5.3 Limitations

We are direct about what this work does not show.

Single backbone. All experiments use Claude Opus 4.6. Topology-task alignment patterns might differ with other models. Our prior OrchestraBench work found that secondary backbone results converged on easy tasks but we did not test switching behavior across backbones.

82 tasks. The suite is small by machine learning standards. With 70 hard tasks and 42 multi-phase tasks, our statistical power is limited. The 78.6% success rate has a 95% confidence interval of roughly ± 9.5 points (binomial). Larger-scale evaluation is needed to confirm the effect size.

Phase boundary detection is brittle. Our Euclidean distance heuristic with a fixed threshold is simple but not optimal. It misses gradual transitions and can be fooled by sudden but temporary shifts. Learned change-point detectors (e.g., BOCPD) would likely perform better.

Switching overhead is non-trivial. 800 tokens per switch and 8.2% total overhead means the approach has negative expected value on tasks where switching doesn’t improve outcomes. The cost model helps, but it depends on accurate reward predictions from the bandit, which are themselves uncertain early in execution.

No cross-task learning. Each task starts with the same warm-started bandit. The system does not learn from previous tasks to improve switching decisions on new ones. A meta-learning extension could amortize the exploration cost.

5.4 Future Directions

Three directions seem promising. First, replacing the fixed threshold with a learned phase boundary detector trained on our annotated phase boundaries. Second, extending to a continuous topology space where the system can interpolate between topologies (e.g., a partially hierarchical debate) rather than switching discretely between three options. Third, cross-task transfer of switching policies, where the bandit’s posterior from completed tasks initializes the prior for new tasks of similar type.

6 Conclusion

We presented AdaptOrch-RT, a system that treats orchestration topology as a runtime variable rather than a design-time constant. By estimating D-I-T task-structure characteristics from execution traces and switching topologies at detected phase boundaries, AdaptOrch-RT achieves 78.6% task success on hard OrchestraBench tasks, up from 64.6% for the best static topology and within 11.4 points of the oracle upper bound.

The key finding is not just that switching helps. It is that switching helps specifically when tasks change structure mid-execution, and that the benefit scales with the magnitude of the structural shift.

On multi-phase tasks, the improvement is 23.8 points. On single-phase tasks, the system correctly holds steady.

Three limitations bound our current results: a single backbone, 82 tasks, and a simple phase boundary detector. But the direction is clear. Topology is not a one-time design choice. Tasks evolve, and the orchestration should evolve with them. Future multi-agent frameworks would benefit from treating topology switching as a first-class operation, with standardized state transfer interfaces and learned switching policies that improve across tasks.

References

- Anonymous. Orchestration topology benchmarking: Which multi-agent LLM architecture fits which task? Under review, 2026.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. Why do multi-agent LLM systems fail? In *NeurIPS 2025*, 2025.
- Yufan Dang and Chen Qian. Multi-agent collaboration via evolving orchestration. In *NeurIPS 2025*, 2025.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *ICML 2024*, 2023.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *ICLR 2024*, 2023.
- Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeonhoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae Won Park. MDAgents: An adaptive collaboration of LLMs for medical decision-making. In *NeurIPS 2024*, 2024.
- Yuxing Lu, Yuhao Hu, Xukai Zhao, and Jie Cao. DyTopo: Dynamic topology routing for multi-agent reasoning via semantic matching. arXiv preprint arXiv:2602.06039, 2026.
- Unknown. Learning to route LLMs from bandit feedback. arXiv preprint arXiv:2510.07429, 2025a.
- Unknown. RouteLLM: Learning to route LLMs with preference data. In *ICLR 2025*, 2025b.
- Unknown. TraceCoder: A trace-driven multi-agent framework for automated debugging. In *ICSE 2026*, 2026.
- Hanqing Wang and Yuhang Yao. DIG to heal: Scaling general-purpose agent collaboration via explainable dynamic decision paths. arXiv preprint arXiv:2603.00309, 2026.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *COLM 2024*, 2023.
- Geunbin Yu. AdaptOrch: Task-adaptive multi-agent orchestration in the era of LLM performance convergence. arXiv preprint arXiv:2602.16873, 2026.
- Guibin Zhang and Kun Wang. G-Designer: Architecting multi-agent communication topologies via graph neural networks. In *ICML 2025*, 2025.
- Hanrong Zhou. Multi-agent design: Optimizing agents with better prompts and topologies (MASS). In *ICML 2025*, 2025.